



# Reproducibility & Determinism at Scale

## Seeds and sharding

- Set global and per-rank RNG seeds (Python, NumPy, framework, CUDA)
- Use a deterministic DistributedSampler so each rank gets a stable, unique shard

## Deterministic ops

- Enable deterministic backends/flags
- Avoid or replace known nondeterministic kernels; document any exceptions

## Floating-point non-associativity

- Different reduction orders/bucketization yield small numeric diffs
- Validate with tolerances (e.g., atol/rtol), not strict bitwise equality

## Version and environment pinning

- Record container image, CUDA/cuDNN/NCCL, driver, framework versions
- Log code commit, configs/hyperparams, hardware topology and world size

## Checkpoints

- Save model weights, optimizer state, RNG states (CPU/CUDA), and dataloader epoch/offset
- Include sharding metadata (DP/TP/PP groups, FSDP/ZeRO maps) for exact restarts
- Support elastic/changed world sizes with remapping info

# Fault Tolerance, Checkpointing & Orchestration

|                          |                                                                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Checkpointing:           | <ul style="list-style-type: none"><li>- Save model, optimizer, LR scheduler, RNG states (CPU/CUDA), and sharding/group metadata</li><li>- Write atomically to durable storage; verify and keep rolling N checkpoints</li></ul>                                                                |
| Failure handling:        | <ul style="list-style-type: none"><li>- Synchronous jobs: relaunch and load latest checkpoint</li><li>- Elastic jobs: surviving ranks continue; newcomers rendezvous, rebuild groups, and reshard from checkpoint</li></ul>                                                                   |
| Launch/orchestration:    | <ul style="list-style-type: none"><li>- Use SLURM/Kubernetes operators; set rank/world-size via env</li><li>- Topology-aware placement (keep tightly coupled groups on same node/switch)</li><li>- Add health checks, timeouts, exponential backoff, and automatic restart policies</li></ul> |
| LR & batch when elastic: | <ul style="list-style-type: none"><li>- Recompute effective global batch after world-size changes</li><li>- Adjust learning rate accordingly; maintain gradient scaling and loss-scaling consistency</li><li>- Log events and validate convergence after elastic resizes</li></ul>            |